

UNIVERSIDADE DO MINHO

Licenciatura em Engenharia Informática



Computação Gráfica

2025/2026

Fase 4

Normais e Coordenadas de Texturas

GRUPO 22

106804, Alice Isabel Faria Soares

106853, Ana Beatriz Freitas

107365, Beatriz Martins Miranda

107367, João Paulo Batista Azevedo

Maio 2026

Índice

1	Introdução	2
2	Normais e Coordenadas de Textura no <i>Generator</i>	3
2.1	Esfera	3
2.2	Plano	3
2.3	Caixa	3
2.4	Cone	4
2.5	Cilindro	4
2.6	Toro	4
2.7	Superfícies de Bézier	5
3	Iluminação no <i>Engine</i>	6
3.1	Modelo de iluminação	6
3.2	Luzes	6
3.3	Propriedades de material	6
3.4	Luz Colorida	7
4	Carregamento e aplicação de texturas	8
5	<i>Parsing</i> do XML estendido	8
5.1	Especificação do Formato XML	8
6	Testes	10
7	Sistema Solar	14
7.1	Skybox	14
7.2	Iluminação	14
7.3	Texturas e materiais dos corpos celestes	14
7.4	Otimizações	14
8	Conclusão	16

1 Introdução

O presente relatório descreve o trabalho realizado na quarta fase do trabalho prático da unidade curricular de Computação Gráfica, inserida no 2º Semestre do 3º Ano da Licenciatura em Engenharia Informática, na Universidade do Minho.

Esta fase assenta em dois eixos de desenvolvimento. Por um lado, o *generator* foi estendido para calcular, por cada vértice gerado, o respetivo vetor normal e as coordenadas de textura, passando estes valores a ser escritos no ficheiro `.3d` a seguir à posição, no formato já estabelecido na fase anterior. Por outro lado, o *engine* foi atualizado para ativar o sistema de iluminação do OpenGL, ler e configurar fontes de luz a partir do XML, carregar texturas a partir de ficheiros de imagem e aplicar propriedades de material individualmente a cada modelo.

A *scene* de demonstração obrigatória é o sistema solar animado da fase anterior, agora enriquecido com texturização e iluminação.

2 Normais e Coordenadas de Textura no *Generator*

Nas fases anteriores, apenas eram utilizadas as coordenadas de posição de cada vértice. Nesta fase, cada primitiva passou também a incluir o vetor normal calculado e as coordenadas de textura. Como o formato dos vértices já suportava o armazenamento destes atributos, não foi necessária qualquer alteração à estrutura do ficheiro .3d. Assim, os dados de posição, normais e coordenadas de textura passaram simplesmente a ser escritos segundo o formato já definido.

2.1 Esfera

A esfera de raio r , centrada na origem, é parametrizada por ângulo de elevação $\phi \in [-\pi/2, \pi/2]$, varrido em *stacks*, e por ângulo azimutal $\theta \in [0, 2\pi[$, varrido em *slices*. Para uma esfera centrada na origem, a normal em qualquer ponto é simplesmente a posição normalizada pelo raio, sendo por isso idêntica ao versor de posição:

$$n(\phi, \theta) = \begin{pmatrix} \cos \phi \sin \theta \\ \sin \phi \\ \cos \phi \cos \theta \end{pmatrix}$$

Esta propriedade é explorada diretamente na implementação onde os componentes da normal são calculados uma única vez e usados tanto para a normal como para a posição, multiplicada por r , sem custo adicional. As coordenadas de textura são obtidas a partir dos índices de iteração i *stack* e j *slice*:

$$u = \frac{j}{slices}, \quad v = \frac{i}{stacks}$$

2.2 Plano

O plano de lado *length* situa-se no plano XZ, centrado na origem. Todos os vértices partilham a mesma normal apontada para cima:

$$n = (0, 1, 0)$$

As coordenadas de textura são obtidas por interpolação linear dos índices das divisões do plano, ficando naturalmente normalizadas para o intervalo $[0, 1]$:

$$u = \frac{i}{divisions}, \quad v = \frac{j}{divisions}$$

2.3 Caixa

A caixa é composta por seis faces independentes, cada uma com uma normal constante $\mathbf{n}$ direcionada para o exterior, correspondendo aos versores axiais $\pm \mathbf{e}_x$, $\pm \mathbf{e}_y$ e $\pm \mathbf{e}_z$:

$$n_{\text{topo}} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad n_{\text{fundo}} = \begin{pmatrix} 0 \\ -1 \\ 0 \end{pmatrix}, \quad n_{\text{frente}} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad n_{\text{trás}} = \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix}, \quad n_{\text{dir}} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad n_{\text{esq}} = \begin{pmatrix} -1 \\ 0 \\ 0 \end{pmatrix}.$$

As coordenadas de textura (u, v) são calculadas diretamente a partir dos índices de subdivisão i e j da grelha de cada face, escalados para o intervalo $[0, 1]$:

$$u = \frac{i}{divisions}, \quad v = \frac{j}{divisions},$$

onde $i, j \in [0, divisions]$.

2.4 Cone

A normal da superfície lateral depende do ângulo θ de cada *slice* e da inclinação da geratriz, sendo calculada por:

$$n(\theta) = \begin{pmatrix} \cos A \cdot \sin \theta \\ \sin A \\ \cos A \cdot \cos \theta \end{pmatrix}, \quad \text{onde} \quad \sin A = \frac{\text{radius}}{\text{slant}}, \quad \cos A = \frac{\text{height}}{\text{slant}}, \quad \text{slant} = \sqrt{\text{radius}^2 + \text{height}^2}.$$

A base do cone possui uma normal constante descendente no eixo Y:

$$n_{\text{base}} = (0, -1, 0).$$

As coordenadas de textura (u, v) para os vértices da lateral utilizam os índices de geração j (*slice*) e i (*stack*), escalados para o intervalo $[0, 1]$ através de $t_0 = i/\text{stacks}$:

$$u_{\text{lateral}} = \frac{j}{\text{slices}}, \quad v_{\text{lateral}} = t_0.$$

Para os vértices da base, as coordenadas de textura são geradas por projeção polar a partir do centro do mapa de texturas:

$$u_{\text{base}} = 0.5 + \sin \theta * 0.5, \quad v_{\text{base}} = 0.5 + \cos \theta * 0.5.$$

2.5 Cilindro

A normal da superfície lateral é puramente radial no plano XZ, variando com o ângulo θ de cada *slice*. As tampas possuem normais constantes paralelas ao eixo Y:

$$n_{\text{lateral}}(\theta) = \begin{pmatrix} \sin \theta \\ 0 \\ \cos \theta \end{pmatrix}, \quad n_{\text{topo}} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad n_{\text{fundo}} = \begin{pmatrix} 0 \\ -1 \\ 0 \end{pmatrix}.$$

As coordenadas de textura (u, v) da lateral mapeiam linearmente os índices de iteração j (*slice*) e i (*stack*):

$$u_{\text{lateral}} = \frac{j}{\text{slices}}, \quad v_{\text{lateral}} = \frac{i}{\text{stacks}}.$$

Nas tampas (topo e fundo), as coordenadas de textura são obtidas por projeção a partir do centro do espaço de textura, tirando partido das componentes radiais:

$$u_{\text{tampas}} = 0.5 + \frac{\sin \theta}{2}, \quad v_{\text{tampas}} = 0.5 + \frac{\cos \theta}{2}.$$

2.6 Toro

O toro é parametrizado utilizando os ângulos ϕ (rotação em torno do eixo Y controlado por *rings*) e θ (rotação em torno do tubo circular controlado por *sides*). A normal unitária em cada ponto da superfície aponta radialmente para o exterior a partir do centro do tubo, sendo calculada por:

$$n(\phi, \theta) = \begin{pmatrix} \cos \theta \cdot \cos \phi \\ \sin \theta \\ \cos \theta \cdot \sin \phi \end{pmatrix}.$$

As coordenadas de textura (u, v) são obtidas mapeando linearmente os índices de iteração i (para as *rings*) e j (para as *sides*) diretamente para o intervalo $[0, 1]$:

$$u = \frac{i}{rings}, \quad v = \frac{j}{sides},$$

onde $i \in [0, rings]$ e $j \in [0, sides]$.

2.7 Superfícies de Bézier

As normais em cada ponto da superfície do *patch* são obtidas através do produto vetorial das suas derivadas parciais. De forma a garantir a concordância com a orientação dos triângulos gerados no algoritmo, efetua-se o produto de $\frac{\partial p}{\partial v}$ por $\frac{\partial p}{\partial u}$:

$$\mathbf{n}(u, v) = \text{normalizar} \left(\frac{\partial p}{\partial v}(u, v) \times \frac{\partial p}{\partial u}(u, v) \right).$$

As componentes das derivadas parciais são calculadas combinando os pontos de controlo P_{ij} com os polinómios de Bernstein $B(t)$ e as respetivas derivadas $B'(t)$:

$$\frac{\partial p}{\partial u}(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B'_i(u) B_j(v) P_{ij}, \quad \frac{\partial p}{\partial v}(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) B'_j(v) P_{ij}.$$

As coordenadas de textura correspondentes a cada vértice coincidem exatamente com os parâmetros de avaliação $(u, v) \in [0, 1]^2$ utilizados no domínio do *patch*:

$$u = u_i \times step, \quad v = v_i \times step,$$

onde $step = \frac{1.0}{tess}$ e os índices $u_i, v_i \in [0, tess]$ mapeiam a discretização da grelha.

3 Iluminação no *Engine*

3.1 Modelo de iluminação

O *engine* adota o modelo de reflexão de *Phong* via pipeline de função fixa do OpenGL, ativado globalmente com `GL_LIGHTING` e `GL_NORMALIZE` (para correção de normais após escalas com `glScalef`). À escala global, é definida uma base de iluminação indireta homogénea branca através de `glLightModelfv(GL_LIGHT_MODEL_AMBIENT, ...)`.

Em cada *frame*, o *engine* configura consecutivamente até 8 fontes de luz (`GL_LIGHT0` a `GL_LIGHT7`). A configuração de cada fonte é realizada imediatamente após a transformação de visualização da câmara (`gluLookAt`), garantindo que o OpenGL converta as coordenadas das luzes para o *eye space* (espaço da câmara) e assegure o cálculo correto da iluminação em cenas com uma câmara dinâmica.

3.2 Luzes

São suportados três tipos de fonte de luz, diferenciados pelo atributo `type`:

- **Pontual** (`type="point"`): emite luz em todas as direções a partir de uma posição (p_x, p_y, p_z) . Configurada com `GL_POSITION` usando o vetor $(p_x, p_y, p_z, 1.0)$, onde a componente homogénea $w = 1$ indica posição finita.
- **Direcional** (`type="directional"`): simula uma fonte infinitamente distante com direção constante (d_x, d_y, d_z) . Configurada com `GL_POSITION` usando $(d_x, d_y, d_z, 0.0)$, onde $w = 0$ posiciona a fonte no infinito.
- **Spot** (`type="spot"`): fonte com posição, direção e ângulo de abertura θ_c (*cutoff*). Além da posição (como na pontual), são definidos `GL_SPOT_DIRECTION` e `GL_SPOT_CUTOFF`.

3.3 Propriedades de material

As propriedades de material de cada modelo são lidas do XML e aplicadas com `glMaterialfv` imediatamente antes do seu *draw call*. O comportamento do material está mapeado em quatro componentes fundamentais do modelo de *Phong*:

- **Difuso** (`GL_DIFFUSE`): cor do objeto sob iluminação direta;
- **Ambiente** (`GL_AMBIENT`): resposta à componente de iluminação global indireta;
- **Especular** (`GL_SPECULAR`): intensidade do reflexo especular;
- **Emissivo** (`GL_EMISSION`): luz própria do objeto, independente de qualquer fonte externa.

O refinamento do brilho é controlado pelo coeficiente s (*shininess*), aplicado à face frontal da geometria com o comando `GL_SHININESS`. Matematicamente, este expoente comprime ou expande o lóbulo especular: valores elevados resultam num ponto de brilho intensamente focado (superfícies metálicas/polidas), enquanto valores nulos dispersam a reflexão especular de forma homogénea.

Sempre que o ficheiro de configuração omite a definição explícita de cores, o *engine* injeta uma assinatura de material cinzenta padrão por omissão (Tabela 1):

Componente	R	G	B	A
Difuso	200	200	200	255
Ambiente	50	50	50	255
Especular	0	0	0	255
Emissivo	0	0	0	255
Shininess		0		

Tabela 1: Valores por omissão das propriedades de material

Quando uma malha acumula simultaneamente propriedades de material e um mapa de textura bidimensional ativo (`GL_TEXTURE_2D`), o ambiente gráfico encontra-se configurado em modo `GL_MODULATE`. Esta configuração dita que a cor final de cada fragmento seja obtida através do produto escalar direto entre os texels extraídos da imagem e os valores cromáticos calculados pelo modelo de iluminação de *Phong*. Caso o bloco de cor ou a textura sejam omitidos, o modelo renderiza exclusivamente com a propriedade que estiver disponível.

3.4 Luz Colorida

Como funcionalidade extra, o *engine* suporta luzes coloridas configuradas diretamente no XML. As cores definidas afetam as seguintes propriedades de cada fonte:

- **Difuso** (`GL_DIFFUSE`): Modula a tonalidade projetada na superfície consoante o ângulo de incidência.
- **Especular** (`GL_SPECULAR`): Altera a cor do ponto de brilho intenso refletido na direção da câmara.

Para evitar a saturação descontrolada da cena, a componente ambiente individual (`GL_AMBIENT`) de cada luz é mantida a preto absoluto (0, 0, 0, 1), delegando a iluminação de preenchimento ao modelo ambiente global do *engine*.

Como exemplo, foi criada a cena `colored_lights.xml`, que recorre a três luzes pontuais com cores distintas. Cada fonte injeta a sua cor própria nos canais difuso e especular, permitindo validar facilmente se a modulação cromática está a ser aplicada de forma correta no modelo de *Phong*.

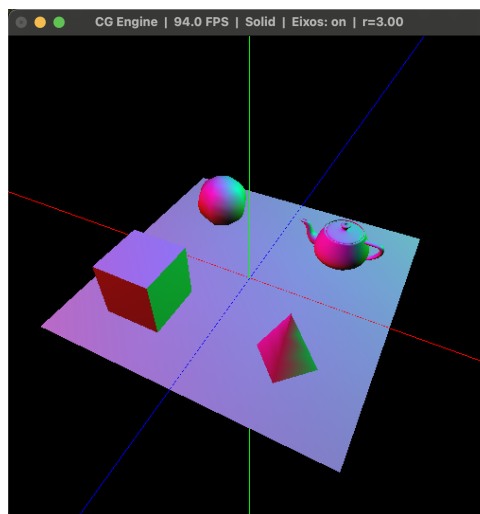


Figura 1: Cena de exemplo de uso das luzes coloridas

4 Carregamento e aplicação de texturas

As texturas são carregadas a partir de ficheiros de imagem durante a execução da aplicação. Cada textura é identificada pelo nome do ficheiro e armazenada num mapa estático, evitando carregamentos redundantes. Assim, pedidos subsequentes à mesma textura reutilizam o identificador já existente na GPU.

A associação entre texturas e modelos é realizada através da tag `texture<>` no ficheiro XML. O parser resolve o caminho relativo à *scene* e guarda o nome do ficheiro na malha. No *renderer*, a textura é carregada sob pedido e registada no mesmo mapa global, garantindo que diferentes malhas que utilizem a mesma imagem partilhem o mesmo identificador OpenGL.

O processo de carregamento segue quatro passos:

1. Leitura e decodificação do ficheiro de imagem em memória;
2. Geração de um identificador de textura com `glGenTextures`;
3. Configuração dos parâmetros de filtragem (`GL_LINEAR`) e de envolvimento (`GL_REPEAT`);
4. Transferência dos dados para a GPU com `glTexImage2D`.

Durante a renderização, caso a malha possua textura associada, o *engine* ativa `GL_TEXTURE_2D`, realiza o *bind* do respetivo identificador e fornece as coordenadas de textura (u,v) geradas pelo *generator* através do `GL_TEXTURE_COORD_ARRAY`. O modo `GL_MODULATE` garante que a cor final da superfície resulte da multiplicação direta entre a cor da imagem e a iluminação calculada pelo modelo de *Phong*. Imediatamente após o *draw call*, o estado do pipeline é desativado para evitar efeitos sobre geometrias subsequentes.

5 Parsing do XML estendido

O módulo de leitura foi estendido para expandir o grafo de cena e suportar os novos requisitos do sistema através de três vertentes:

1. **Iluminação:** Deteta o bloco `<lights>`, extraindo as propriedades geométricas e cromáticas das fontes de luz para uma lista global na cena.
2. **Texturas:** Deteta a tag `<texture>` nos modelos e utiliza um sistema de *caching* (tabelas de dispersão) para evitar uploads redundantes para a GPU.
3. **Materiais:** Processa os coeficientes e cores descritos na tag `<color>` de cada modelo, associando-os individualmente à respetiva malha.

O *engine* garante retrocompatibilidade: na ausência destas tags, são aplicados automaticamente os estados padrão.

5.1 Especificação do Formato XML

A configuração global das fontes de luz suporta a definição geométrica e a especificação opcional de cor (*Luz Colorida*):

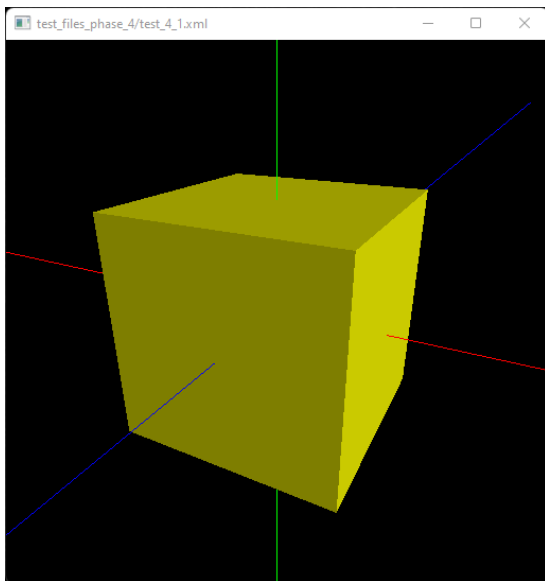
```
<lights>
  <light type="point" posX="0" posY="0" posZ="0">
    <color R="255" G="0" B="0" />
  </light>
  <light type="directional" dirX="1" dirY="1" dirZ="1" />
  <light type="spot" posX="0" posY="10" posZ="0"
    dirX="0" dirY="-1" dirZ="0" cutoff="45" />
</lights>
```

Para associar texturas e propriedades óticas de material a um modelo, a estrutura respeita a seguinte hierarquia:

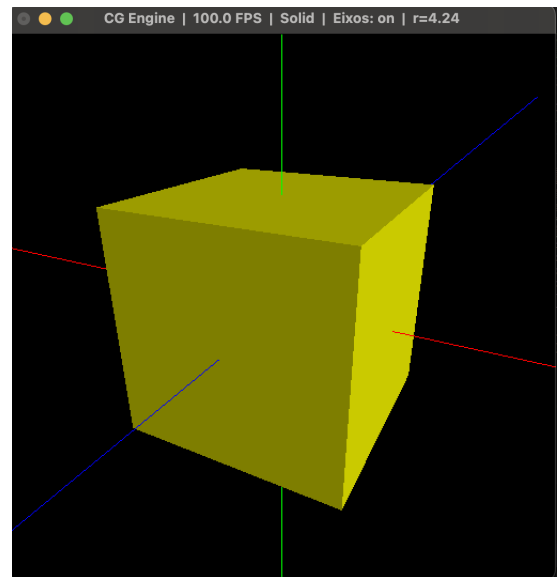
```
<model file="sphere.3d">
  <texture file="earth.jpg" />
  <color>
    <diffuse R="200" G="200" B="200" />
    <ambient R="50" G="50" B="50" />
    <specular R="0" G="0" B="0" />
    <emissive R="0" G="0" B="0" />
    <shininess value="0" />
  </color>
</model>
```

6 Testes

Nesta secção são apresentados os resultados obtidos para os testes da Fase 4, comparados com os resultados esperados fornecidos pela equipa docente.

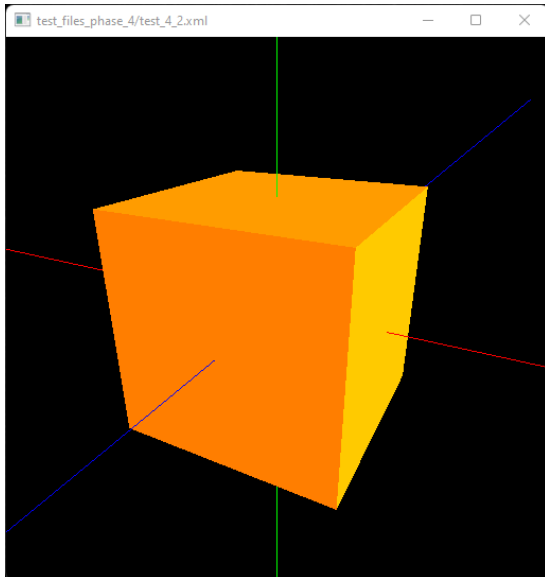


(a) Resultado Esperado

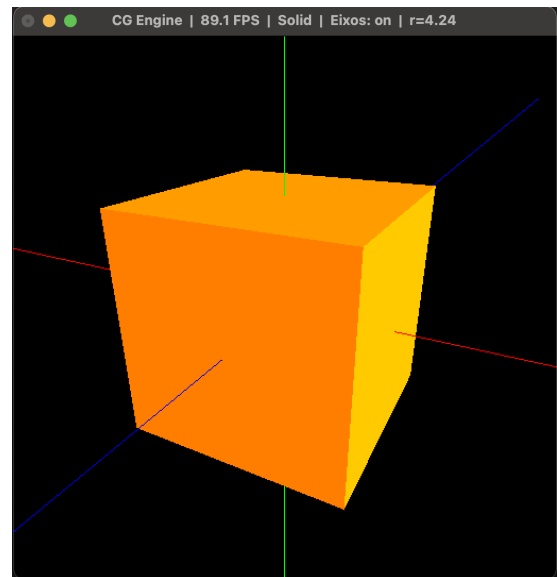


(b) Resultado Obtido

Figura 2: Teste 1

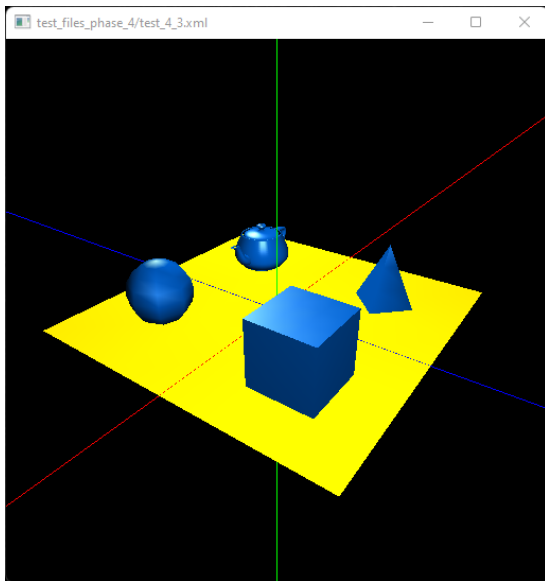


(a) Resultado Esperado

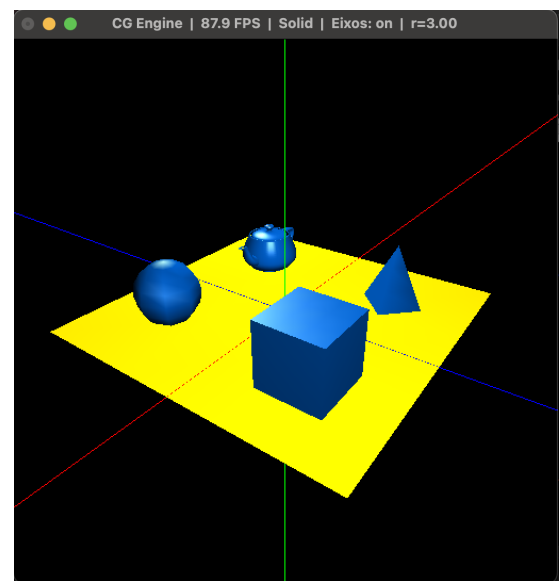


(b) Resultado Obtido

Figura 3: Teste 2

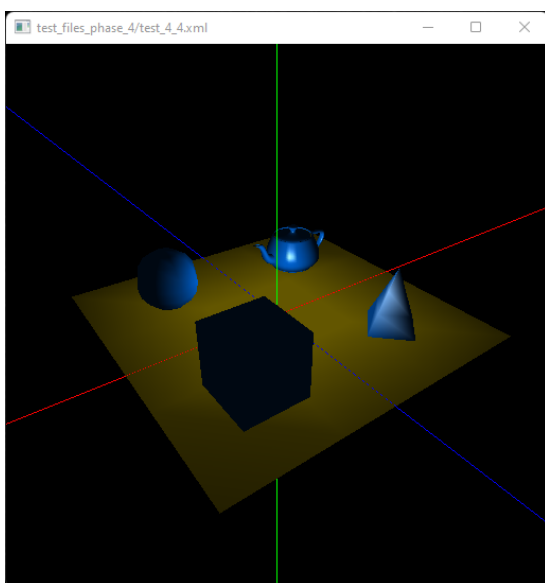


(a) Resultado Esperado

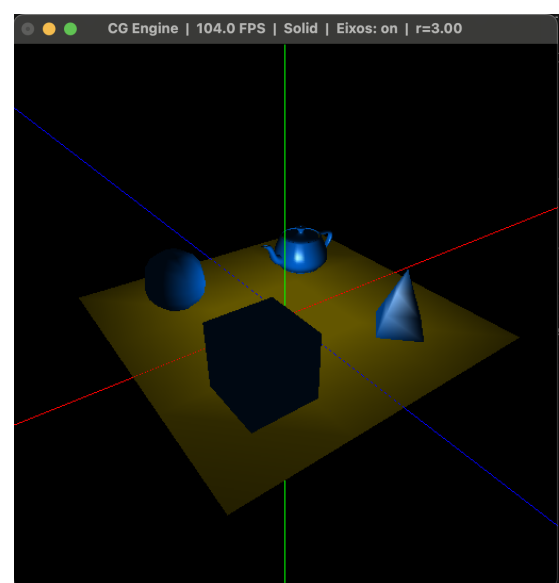


(b) Resultado Obtido

Figura 4: Teste 3

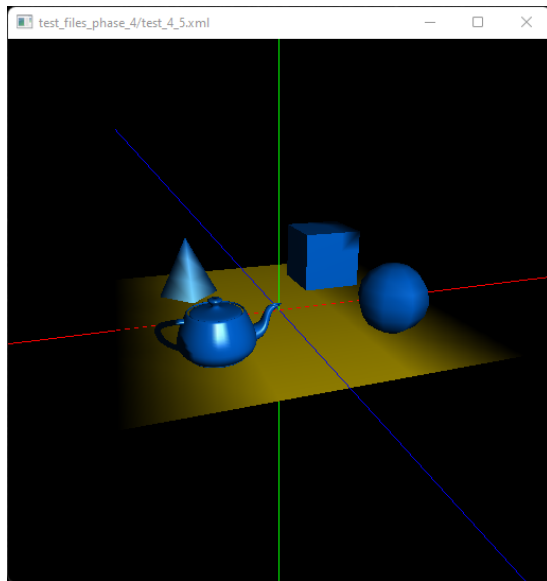


(a) Resultado Esperado

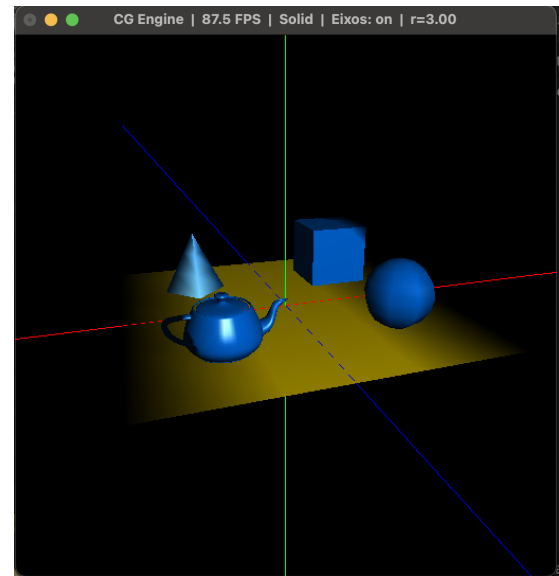


(b) Resultado Obtido

Figura 5: Teste 4

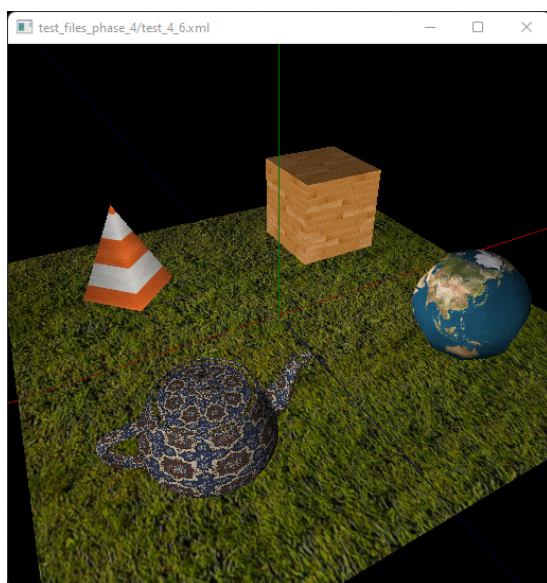


(a) Resultado Esperado

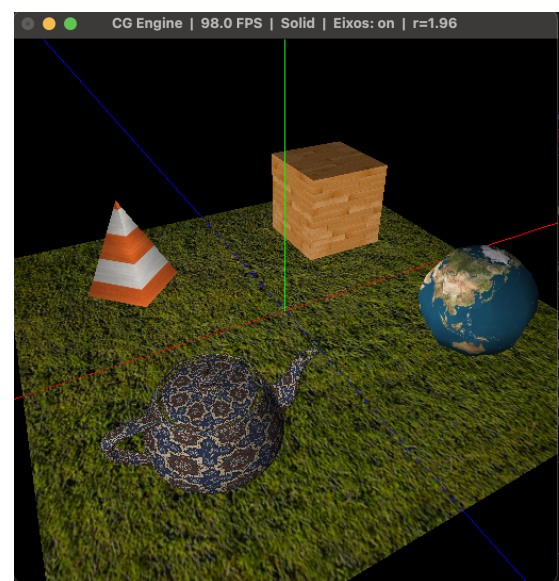


(b) Resultado Obtido

Figura 6: Teste 5

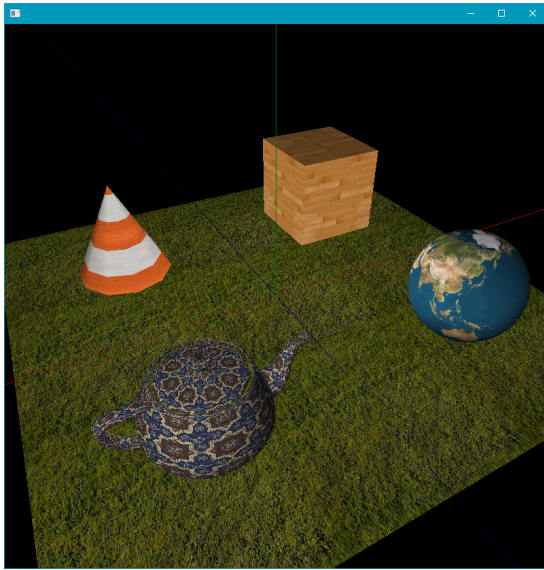


(a) Resultado Esperado

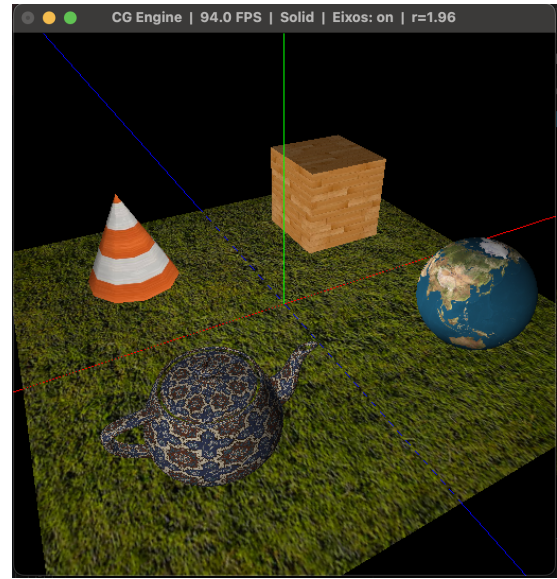


(b) Resultado Obtido

Figura 7: Teste 6



(a) Resultado Esperado



(b) Resultado Obtido

Figura 8: Teste 7

7 Sistema Solar

A *scene* de demonstração desta fase é o sistema solar animado da fase anterior, agora enriquecido com iluminação, texturas individuais e um fundo estelar a funcionar como *skybox* esférica. A hierarquia de grupos, as órbitas de *Catmull-Rom*, as rotações temporais e o cometa de *Bézier* mantêm-se inalterados. As alterações ao ficheiro XML concentram-se na secção `<lights>` e na inclusão das propriedades `<texture>` e `<color>` em cada nó de modelo.

7.1 Skybox

O céu da cena é representado por uma esfera envolvente, modelada através de `sky.3d` e texturada com `8k_stars_milky_way.jpg`. Como o modelo está invertido e envolve toda a cena, o observador tem a sensação de estar imerso no espaço, sem que o fundo interfira com a renderização dos corpos celestes. Na prática, esta solução cumpre o papel de *skybox*, mas com geometria esférica em vez de um cubo.

7.2 Iluminação

O Sol é modelado como uma fonte de luz pontual colocada na origem do referencial do mundo, refletindo a sua posição central no sistema. Existe ainda uma luz direcional adicional, que funciona como preenchimento suave e ajuda a manter visíveis as superfícies menos expostas ao Sol.

```
<lights>
  <light type="point" posx="0" posy="0" posz="0" />
  <light type="directional" dirx="-0.3" diry="1" dirz="0.4" />
</lights>
```

Para que o modelo do Sol apareça sempre completamente visível, independentemente da direção da fonte de luz, foi atribuído um componente emissivo elevado à sua esfera. Esta técnica simula a autoluminescência da estrela sem alterar a iluminação dos restantes corpos.

7.3 Texturas e materiais dos corpos celestes

As texturas dos planetas foram obtidas de repositórios de imagens planetárias de domínio público e associadas individualmente em cada nó do XML. O cometa de *Bézier* complementa a leitura visual da cena, mas não faz parte do conjunto planetário. A cada corpo foi associado um perfil de material ajustado individualmente: o Sol possui emissivo elevado, os planetas têm difuso médio e ambiente baixo, sem emissão própria, e as luas seguem um perfil semelhante ao dos planetas, com difuso ligeiramente inferior.

Corpo	Difuso	Ambiente	Especular	Emissivo
Sol	elevado	elevado	nenhum	elevado
Planetas	médio	baixo	fraco	nenhum
Luas	médio	muito baixo	muito fraco	nenhum
Anel de Saturno	baixo	muito baixo	fraco	nenhum

Tabela 2: Perfis de material por tipo de corpo celeste

7.4 Otimizações

Dado que todos os corpos celestes partilham o mesmo modelo de esfera, a cena beneficia diretamente do mecanismo de *caching* de geometria implementado no *engine*: os VBOs de vértices e de índices

são carregados para a GPU uma única vez e reutilizados pelas 24 instâncias presentes na cena. Após o *upload*, os dados originais são libertados da RAM, eliminando a duplicação entre memória do sistema e GPU. Este mecanismo reduz a transmissão de dados GPU em aproximadamente 96% (1 upload vs. 24 uploads).



Figura 9: Sistema solar com texturização e iluminação — vista geral

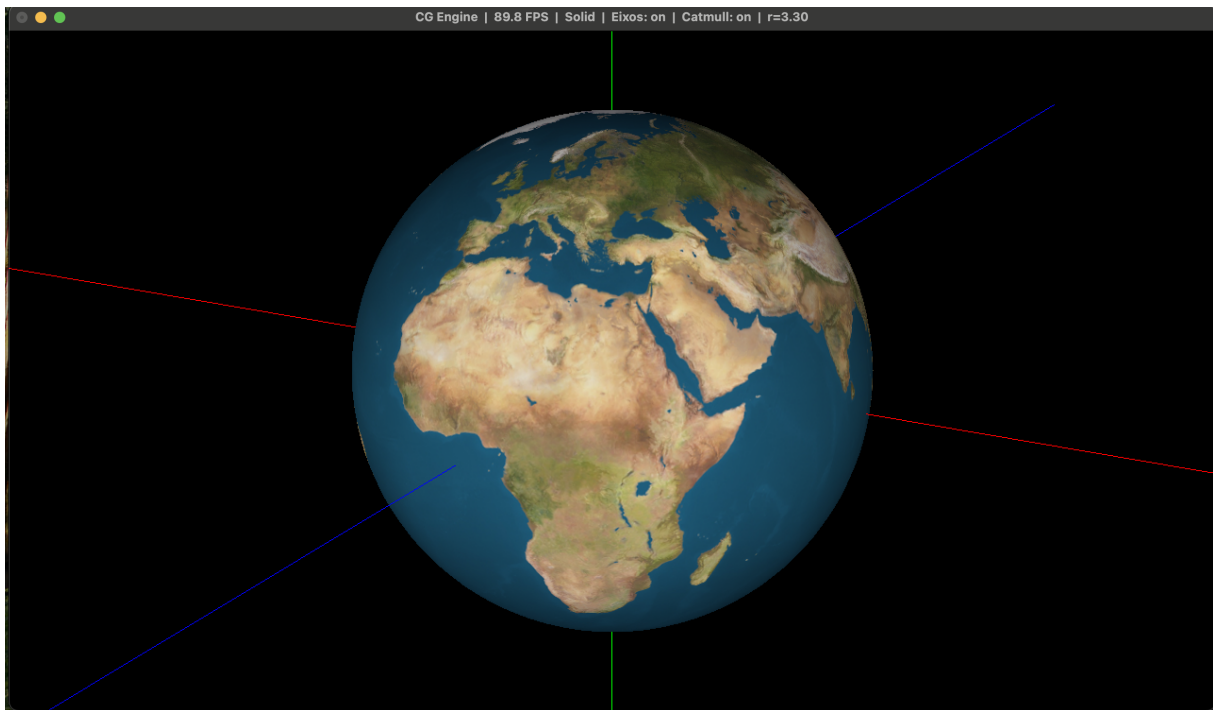


Figura 10: Detalhe da Terra com textura e iluminação difusa

8 Conclusão

Nesta fase foram implementadas com sucesso as duas componentes obrigatórias do projeto. O *generator* foi estendido, garantindo que todas as primitivas passem a exportar normais analíticas exatas e coordenadas de textura mapeadas linearmente para cada vértice.

No *engine*, a integração do modelo de reflexão de *Phong* permitiu o suporte robusto a fontes de luz pontuais, direcionais e focais (*spot*), bem como a aplicação individualizada de propriedades óticas de material e texturas bidimensionais. Como elemento de diferenciação e extra ao projeto, o suporte para luz colorida expandiu o realismo visual das cenas através da modulação cromática dinâmica nos canais difuso e especular. Toda esta infraestrutura foi desenhada com foco na eficiência, recorrendo a VBOs indexados e a um sistema de *caching* de texturas que elimina a redundância na memória de vídeo.

A correta renderização do sistema solar animado validou a natureza modular e a total retrocompatibilidade do motor de jogo desenvolvido ao longo do semestre. Como trabalho futuro, projeta-se a implementação de técnicas de otimização geométrica, nomeadamente o *Frustum Culling*, de forma a descartar antecipadamente da *pipeline* as malhas poligonais que se encontrem fora do volume de visualização da câmara, maximizando assim a taxa de atualização (*framerate*) em cenários de elevada complexidade.